

Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Contents

- ▶ Feature detection (chapter 4) (Richard Szeliski book)
- Template matching
- Corners (Harris Detector)
- Blobs
- Descriptors
- Scale Invariant Feature Transform
- SIFT using opencv library
- SIFT examples

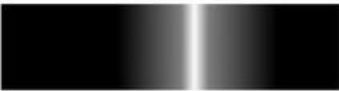
Template matching



Template object

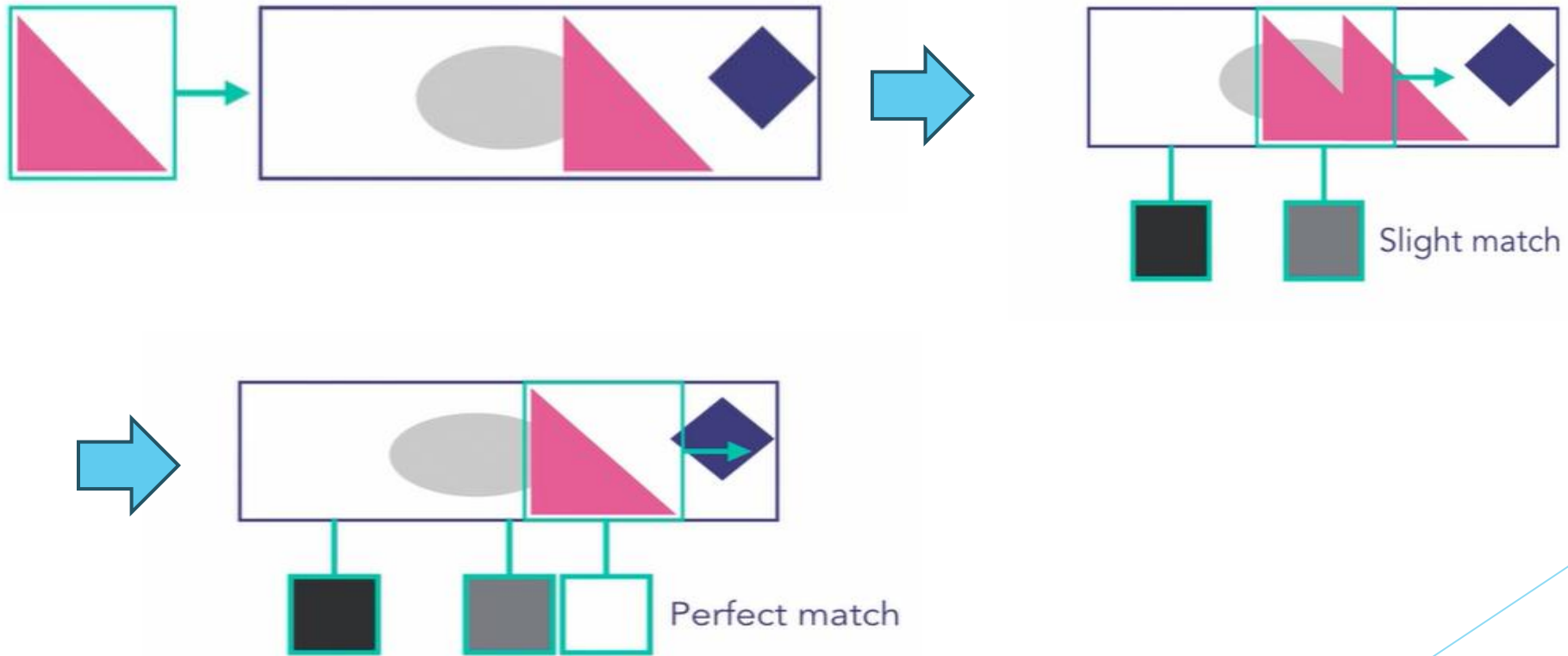


Image containing object



Resulting template match

Template matching



Template matching

Inputs

Frame



Template (scaled)

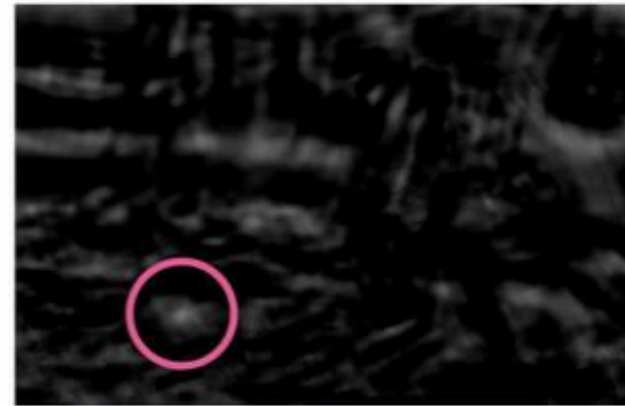


Method

cv2.TM_CCOEFF,
cv2.TM_CCOEFF_NORMED,
cv2.TM_CCORR,
cv2.TM_CCORR_NORMED,
cv2.TM_SQDIFF,
cv2.TM_SQDIFF_NORMED



Output



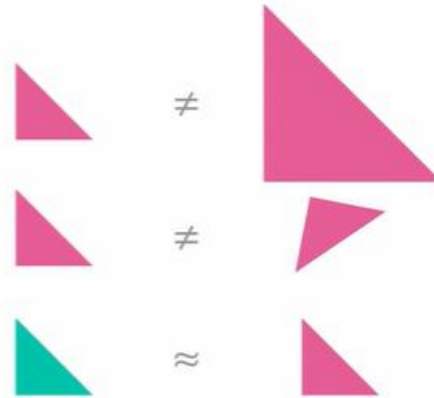
Brightest spot = Best matching x,y

Limitation

Not scale invariant

Not rotation invariant

Requires having a well-formed
template and making input
assumptions



Template matching

```
import numpy as np
import cv2
#from google.colab.patches import cv2_imshow
template = cv2.imread('template.jpg',0)
frame = cv2.imread("players.jpg",0)

cv2.imshow("Frame",frame)
cv2.imshow("Template",template)

result = cv2.matchTemplate(frame, template, cv2.TM_CCOEFF_NORMED)

min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
print(max_val,max_loc)
cv2.circle(result,max_loc, 15,255,2)
cv2.imshow("Matching",result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Template matching



Feature detection

- ▶ Feature detection and matching are an essential component of many computer vision applications.



(a)



(b)



(c)



(d)

Figure 4.1 A variety of feature detectors and descriptors can be used to analyze, describe and match images:

(a) point-like interest operators (Brown, Szeliski, and Winder 2005) IEEE;

(b) region-like interest operators (Matas, Chum, Urban et al. 2004 (c) edges (Elder and Goldberg 2001)

(d) straight lines (Sinha, Steedly, Szeliski et al. 2008)

Feature detection

- ▶ The first kind of feature that you may notice are specific locations in the images, such as mountain peaks, building corners, doorways, or interestingly shaped patches of snow.
- ▶ These kinds of localized feature are often called keypoint features or interest points
- ▶ Another class of important features are edges, e.g., the profile of mountains against the sky. **These kinds of features can be matched based on their orientation and local appearance (edge profiles) and can also be good indicators of object boundaries and occlusion events in image sequences.**

Edges can be grouped into longer curves and straight line segments, which can be directly matched or analyzed to find vanishing points and hence internal and external camera parameters

Motivation: Build a Panorama



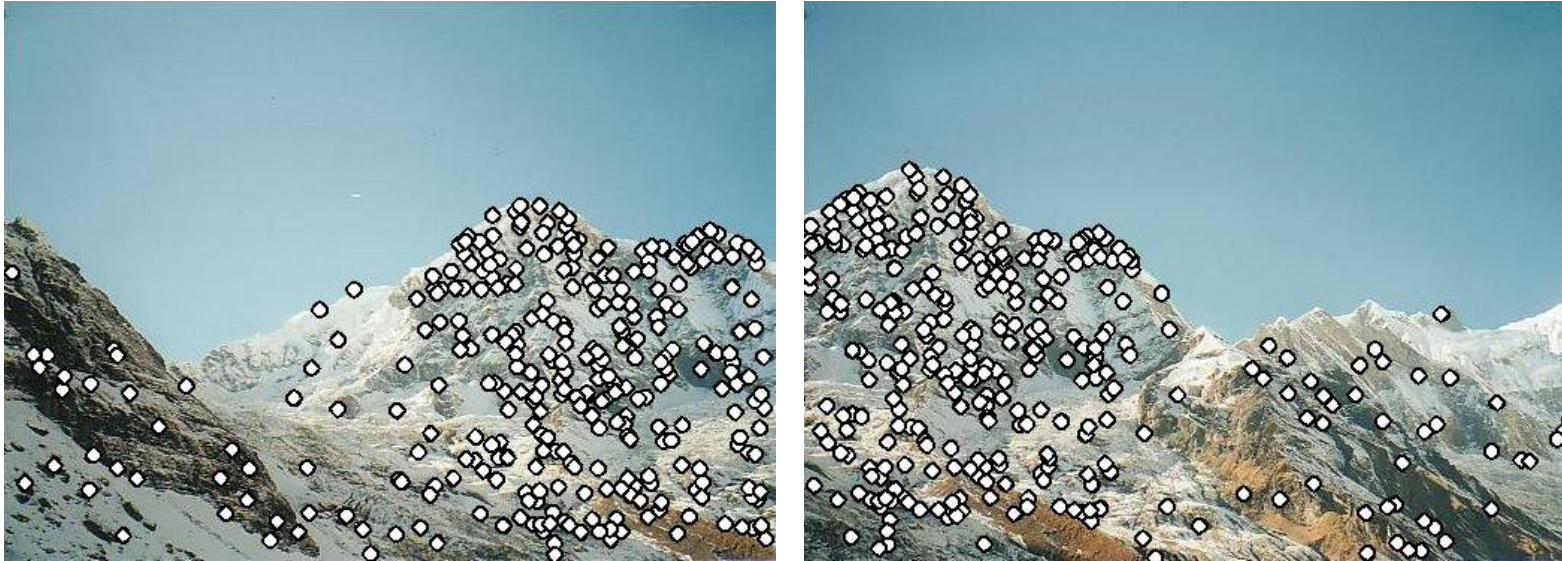
How do we build panorama?

- ▶ We need to match (align) images



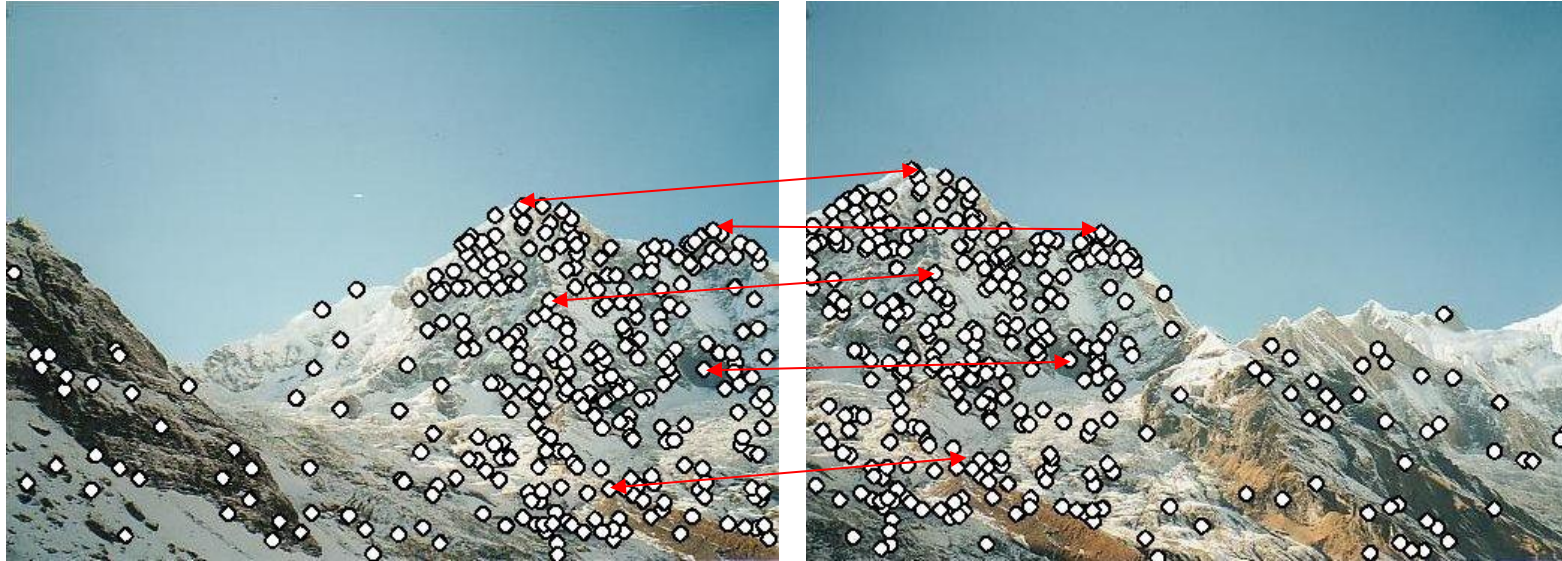
Matching with Features

- Detect feature points in both images



Matching with Features

- Detect feature points in both images
- Find corresponding pairs



Matching with Features

- Detect feature points in both images
- Find corresponding pairs
- Use these pairs to align images



Matching with Features

- ▶ Problem 1:
 - ▶ Detect the *same* point *independently* in both images



no chance to match!

We need a repeatable detector

Matching with Features

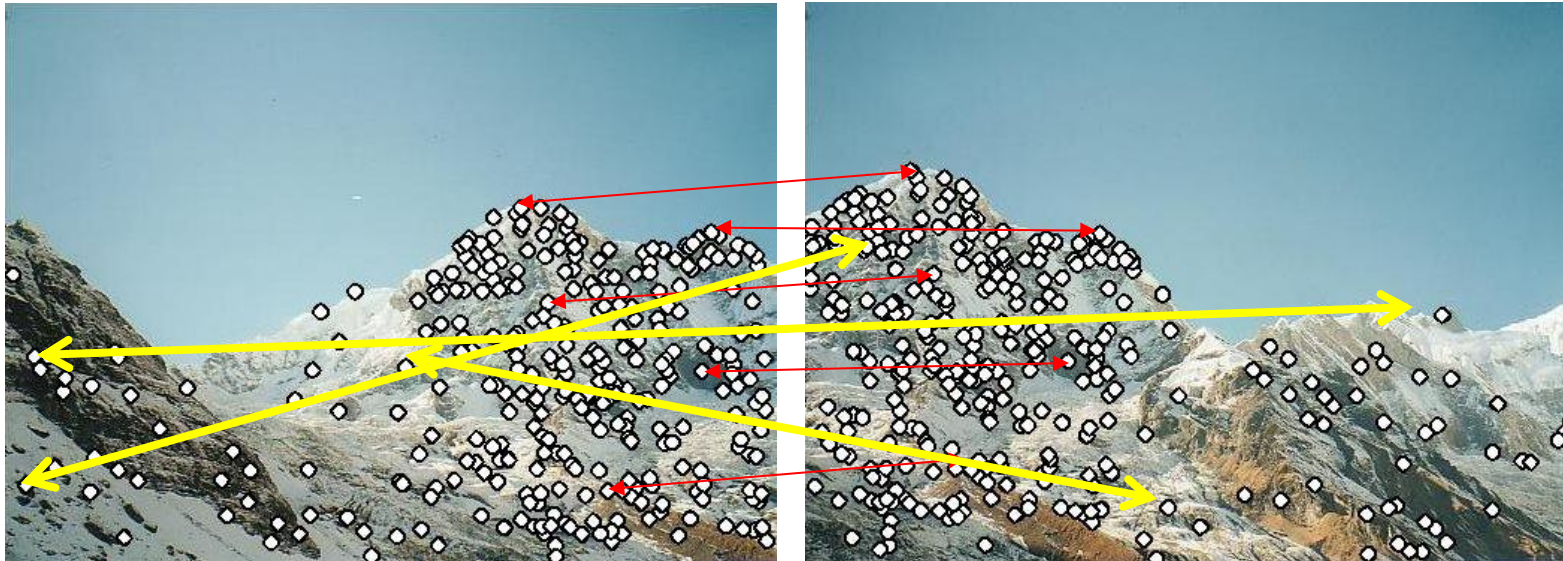
- ▶ Problem 2:
 - ▶ For each point correctly recognize the corresponding one



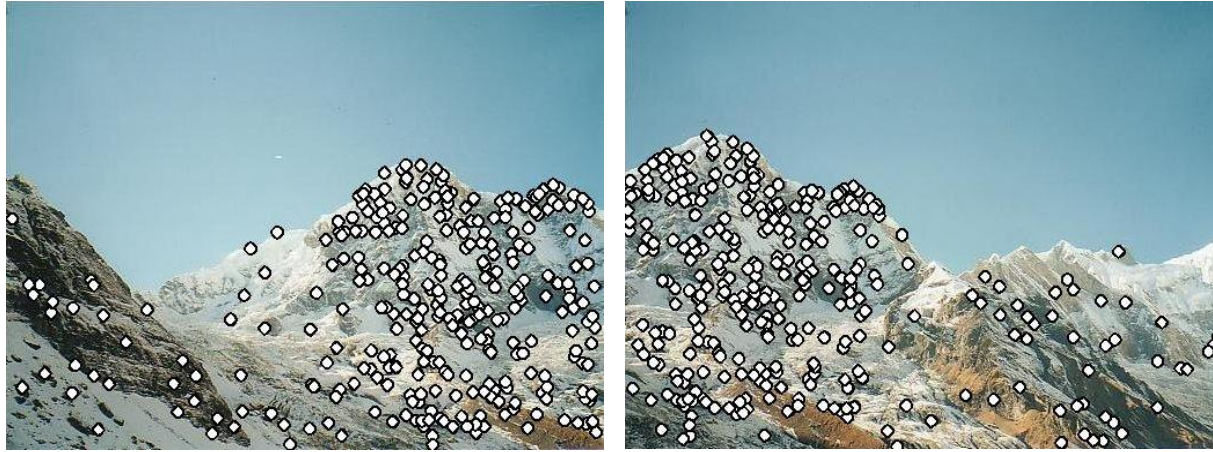
We need a reliable and distinctive descriptor

Matching with Features

- ▶ Problem 3:
 - ▶ Need to estimate transformation between images, despite erroneous correspondences.



Characteristics of good features

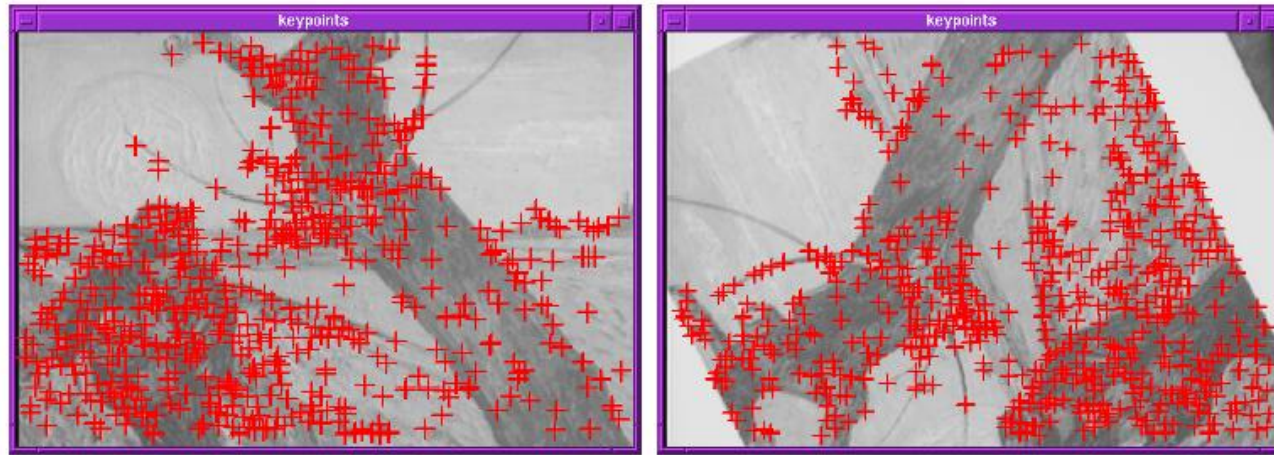


- Repeatability
 - ▶ The same feature can be found in several images despite geometric and photometric transformations
- Saliency
 - ▶ Each feature has a distinctive description
- Compactness and efficiency
 - ▶ Many fewer features than image pixels
- Locality
 - ▶ A feature occupies a relatively small area of the image; robust to clutter and occlusion

Applications

- ▶ Feature points are used for:
 - ❖ Motion tracking
 - ❖ Image alignment
 - ❖ 3D reconstruction
 - ❖ Object recognition
 - ❖ Indexing and database retrieval
 - ❖ Robot navigation

Finding Corners

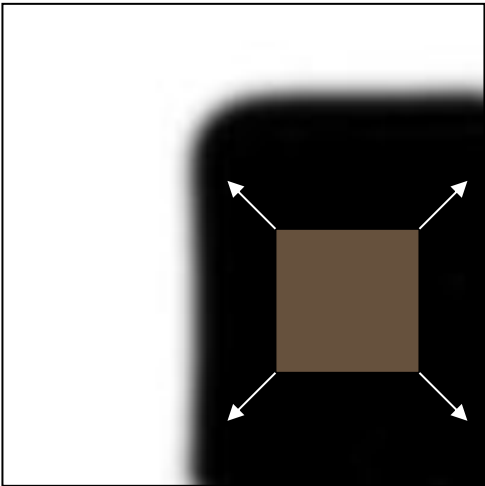


- Key property: in the region around a corner, image gradient has two or more dominant directions
- Corners are repeatable and distinctive

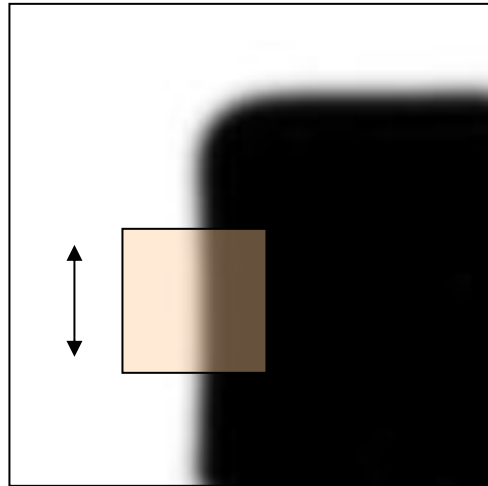
C.Harris and M.Stephens. ["A Combined Corner and Edge Detector."](#)
Proceedings of the 4th Alvey Vision Conference: pages 147--151.

Corner Detection: Basic Idea

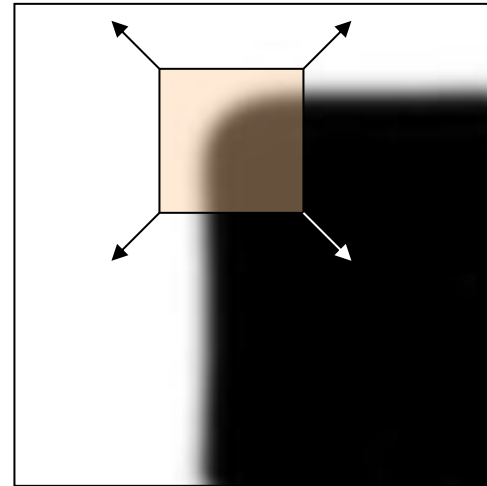
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give a *large change* in intensity



“flat” region:
no change in
all directions



“edge”:
no change
along the edge
direction

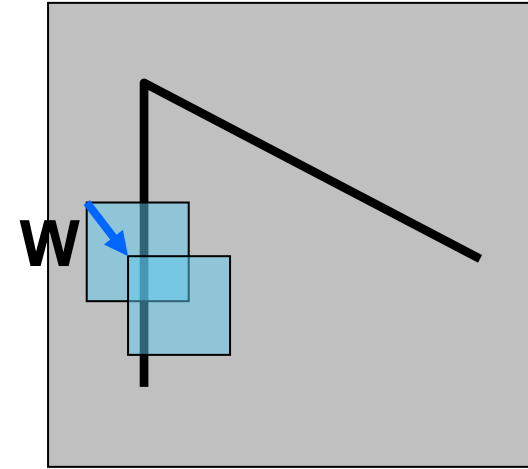


“corner”:
significant
change in all
directions

Feature detection: the math

Consider shifting the window **W** by (u,v)

- how do the pixels in **W** change?
- compare each pixel before and after by summing up the squared differences (SSD)
- this defines an SSD “error” of $E(u,v)$:

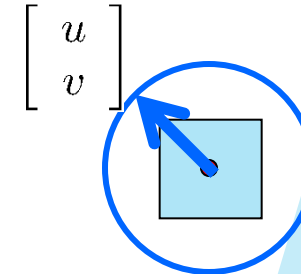
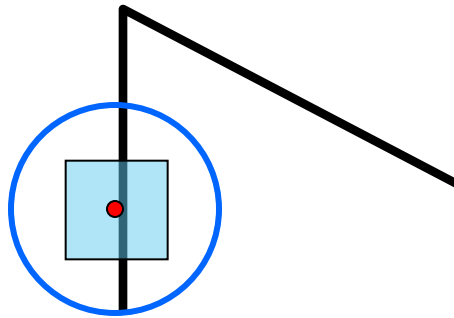


$$E(u, v) = \sum_{(x,y) \in W} [I(x + u, y + v) - I(x, y)]^2$$

Feature detection: the math

This can be rewritten:

$$E(u, v) = \sum_{(x,y) \in W} [u \ v] \underbrace{\begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix}}_H \begin{bmatrix} u \\ v \end{bmatrix}$$



For the example above

- You can move the center of the green window to anywhere on the blue unit circle
- Which directions will result in the largest and smallest E values?
- We can find these directions by looking at the eigenvectors of H

eigenvalue/eigenvector review

The **eigenvectors** of a matrix **A** are the vectors **x** that satisfy:

$$Ax = \lambda x$$

The scalar λ is the **eigenvalue** corresponding to **x**

- ▶ The eigenvalues are found by solving:

$$\det(A - \lambda I) = 0$$

- ▶ In our case, $A = H$ is a 2x2 matrix, so we have

$$\det \begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{21} & h_{22} - \lambda \end{bmatrix} = 0$$

- ▶ The solution:

$$\lambda_{\pm} = \frac{1}{2} \left[(h_{11} + h_{22}) \pm \sqrt{4h_{12}h_{21} + (h_{11} - h_{22})^2} \right]$$

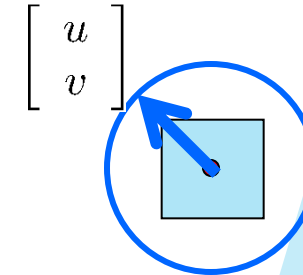
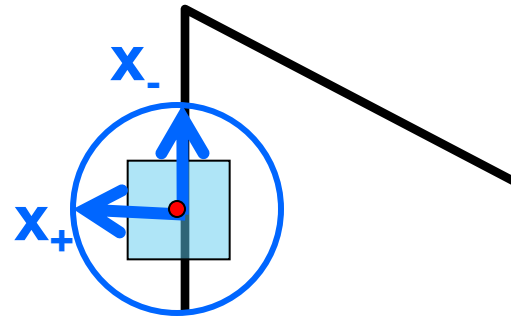
Once you know λ , you find **x** by solving

$$\begin{bmatrix} h_{11} - \lambda & h_{12} \\ h_{21} & h_{22} - \lambda \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$$

Feature detection: the math

This can be rewritten:

$$E(u, v) = \sum_{(x,y) \in W} [u \ v] \underbrace{\begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix}}_H \begin{bmatrix} u \\ v \end{bmatrix}$$



Eigenvalues and eigenvectors of H

- Define shifts with the smallest and largest change (E value)
- x_+ = direction of **largest** increase in E.
- λ_+ = amount of increase in direction x_+
- x_- = direction of **smallest** increase in E.
- λ_- = amount of increase in direction x_+

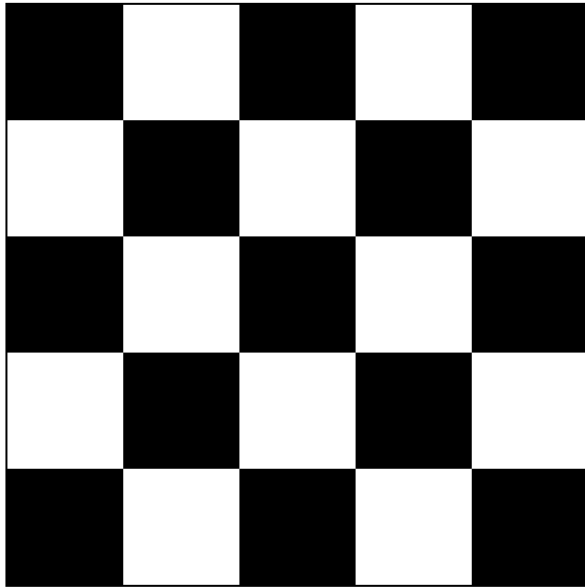
$$Hx_+ = \lambda_+ x_+$$

$$Hx_- = \lambda_- x_-$$

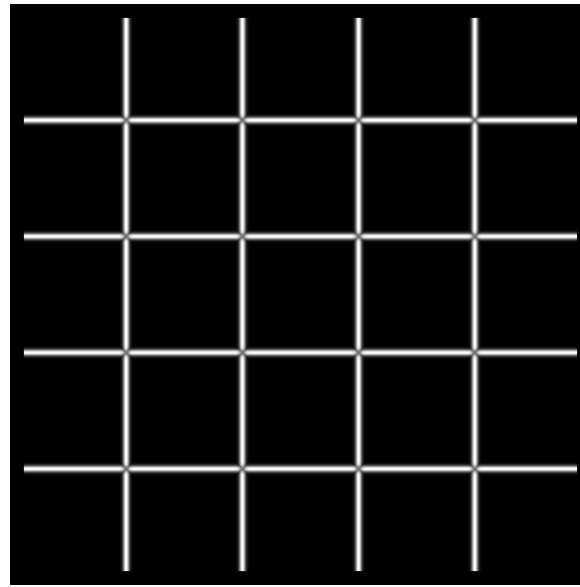
Feature detection summary

Here's what you do

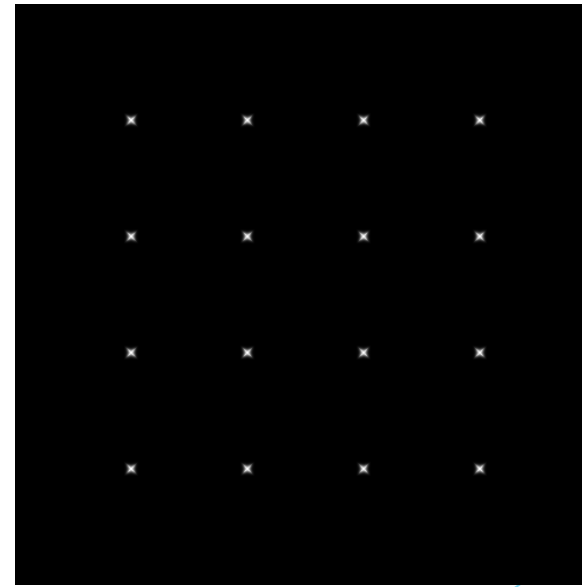
- Compute the gradient at each point in the image
- Create the ***H*** matrix from the entries in the gradient
- Compute the eigenvalues.
- Find points with large response ($\lambda_- > \text{threshold}$)
- Choose those points where λ_- is a local maximum as features



I



λ_+



λ_-

Harris detector: Steps

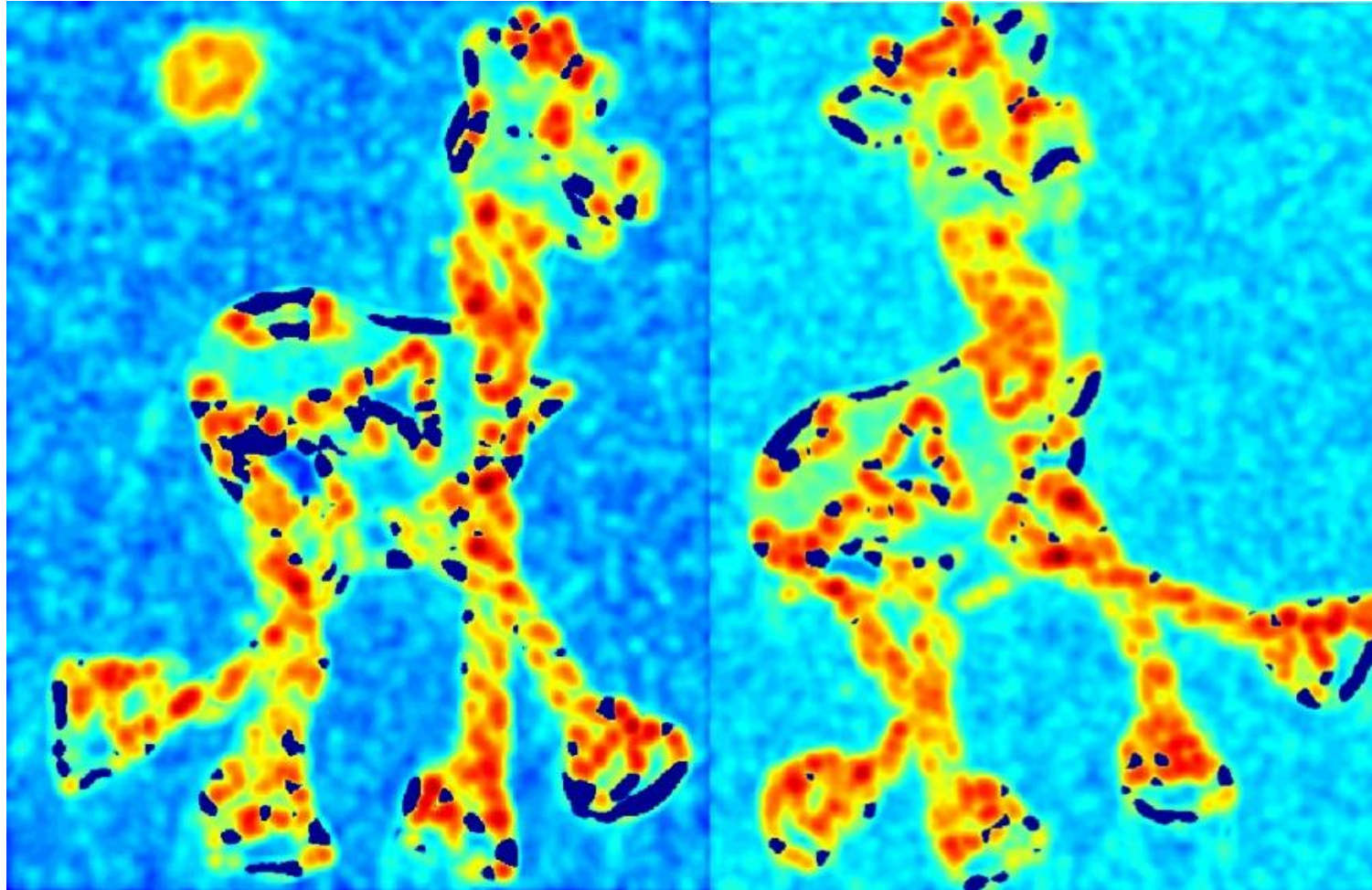
1. Compute Gaussian derivatives at each pixel
2. Compute second moment matrix H in a Gaussian window around each pixel
3. Compute corner response function R
4. Threshold R
5. Find local maxima of response function (nonmaximum suppression)

Harris Detector: Steps



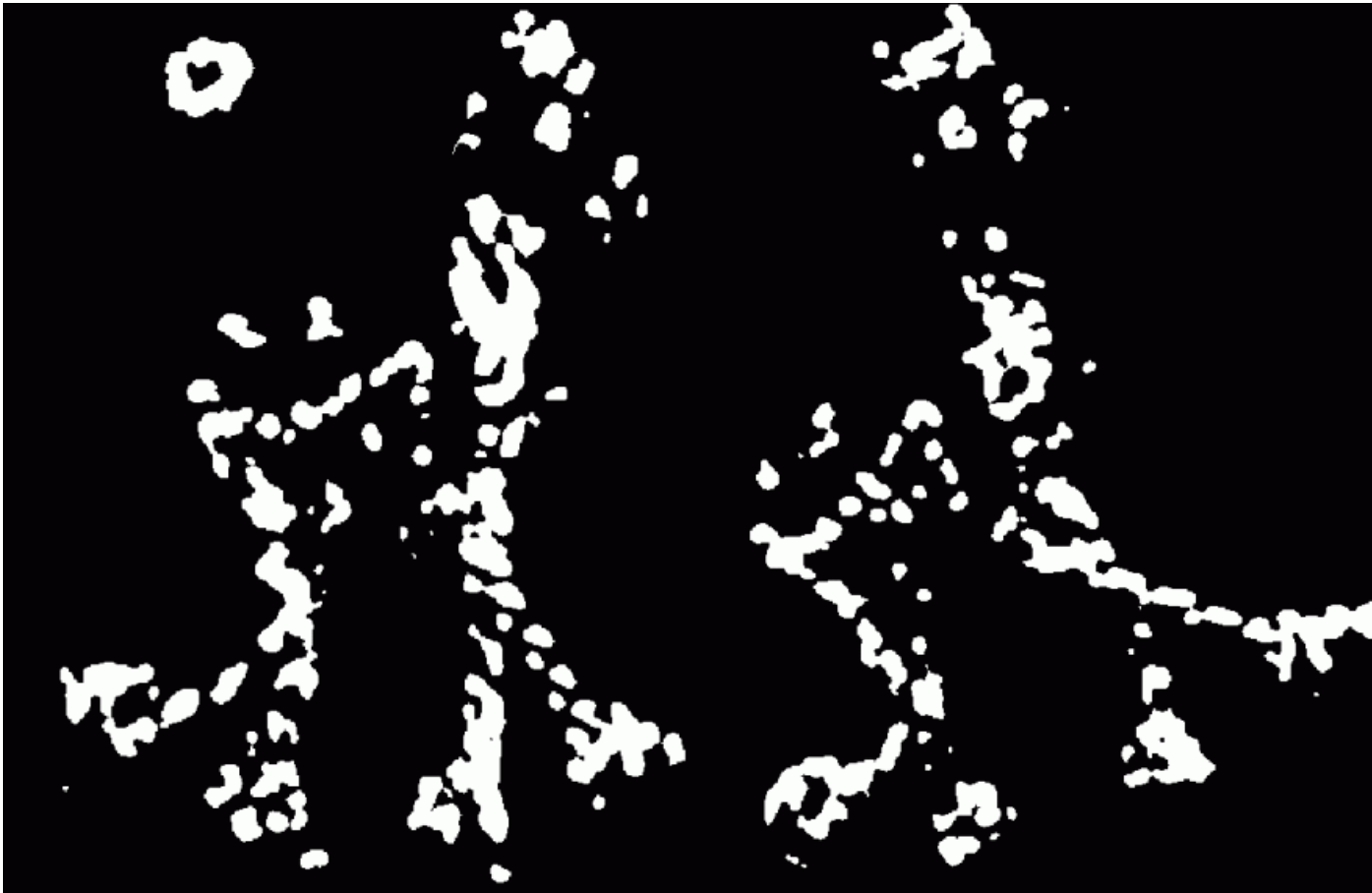
Harris Detector: Steps

Compute corner response R



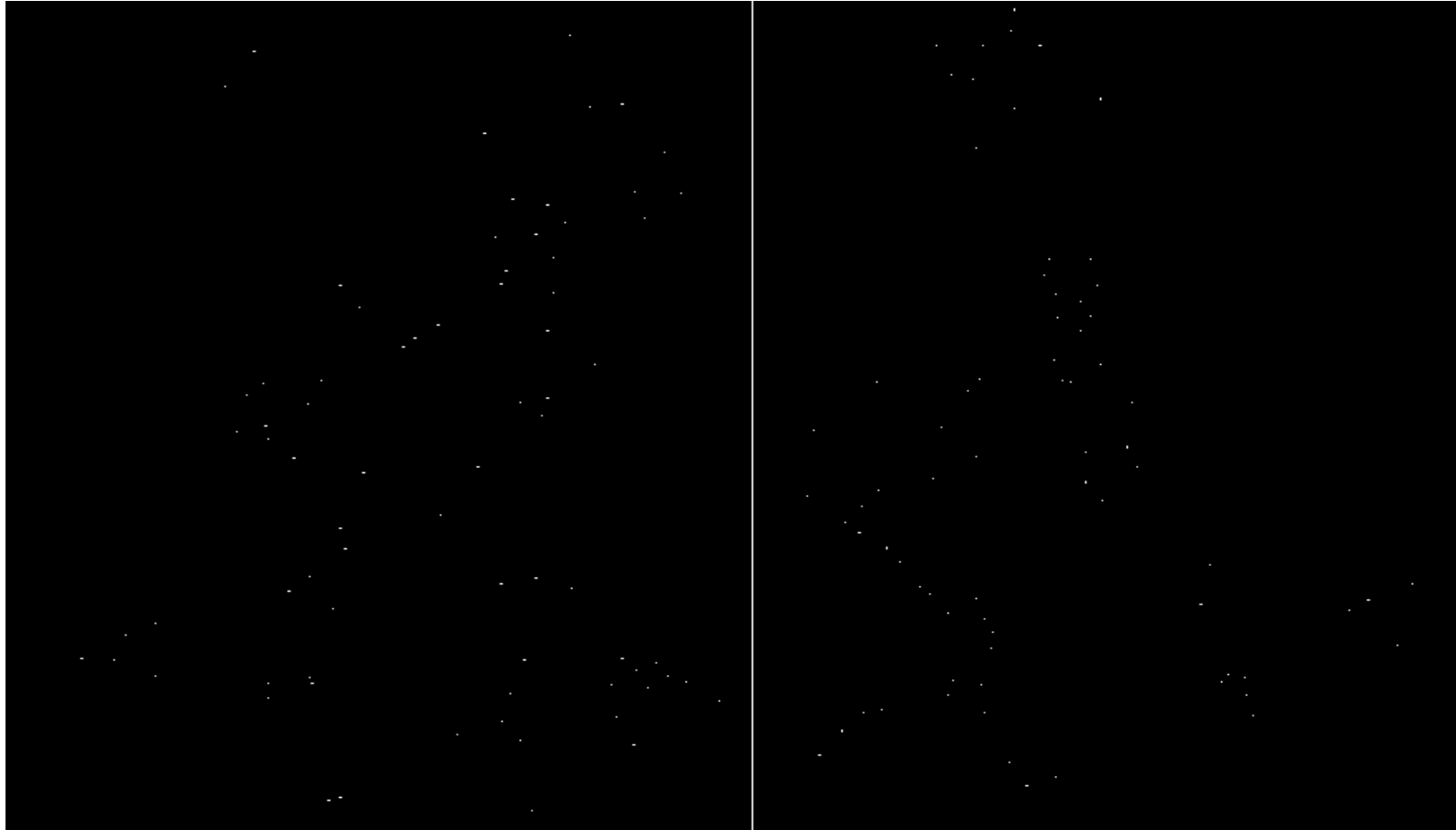
Harris Detector: Steps

Find points with large corner response: $R > \text{threshold}$



Harris Detector: Steps

Take only the points of local maxima of R



Harris Detector: Steps



Invariance and covariance

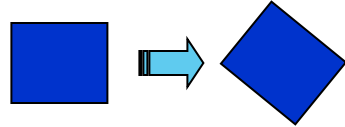
- We want features to be *invariant* to photometric transformations and *covariant* to geometric transformations
 - ▶ **Invariance:** image is transformed and features do not change
 - ▶ **Covariance:** if we have two transformed versions of the same image, features should be detected in corresponding locations



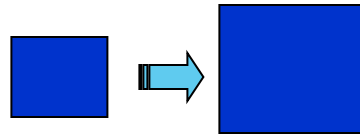
Transformations

- Geometric

- Rotation

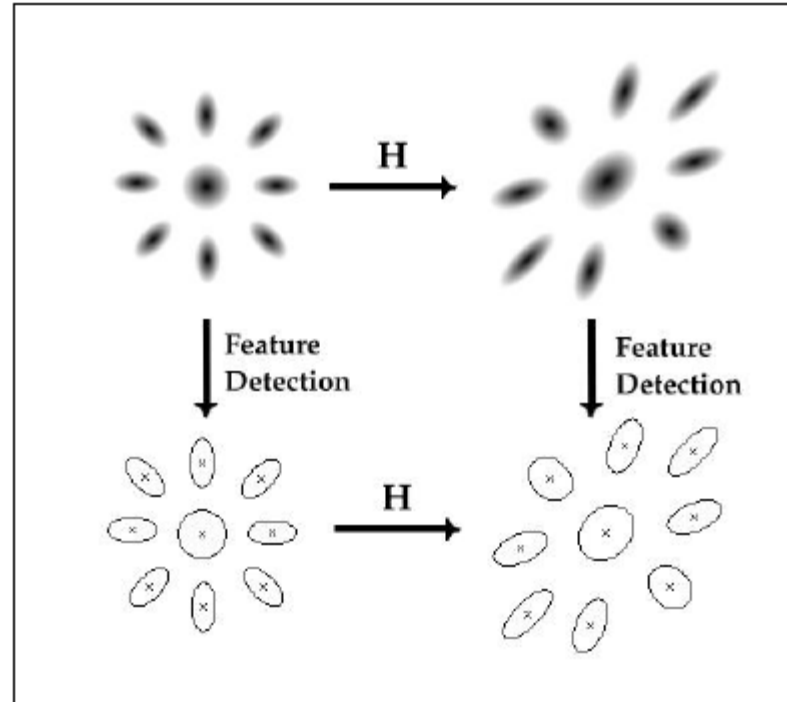
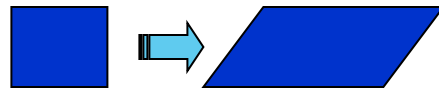


- Scale



- Affine

valid for:
orthographic camera,
locally planar object



T. Kadir, A. Zisserman and M. Brady, An Affine invariant salient region detector, ECCV 2004

- Photometric

- Affine intensity change ($I \rightarrow aI + b$)

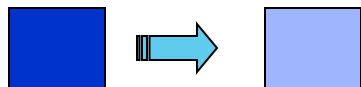
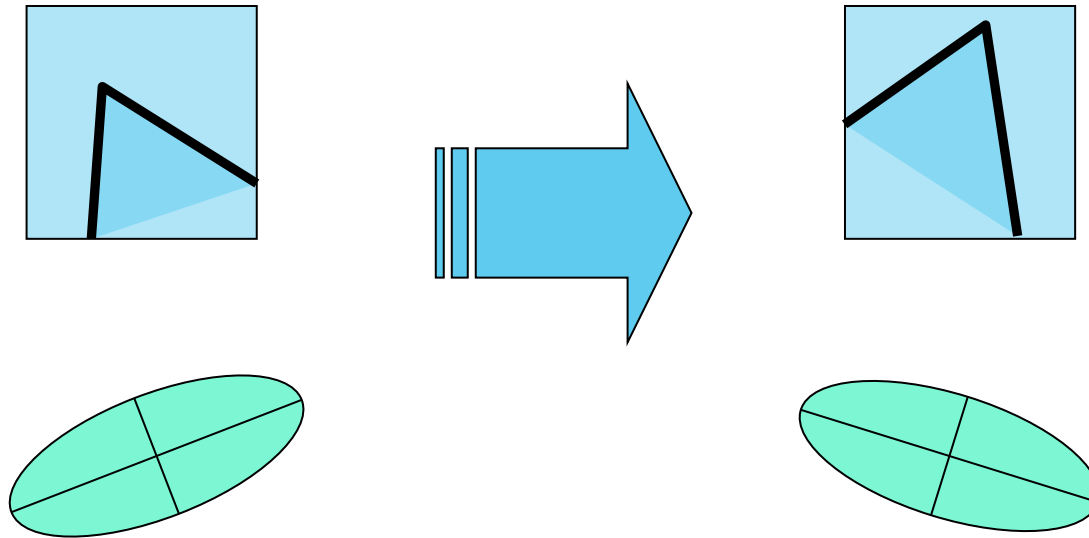


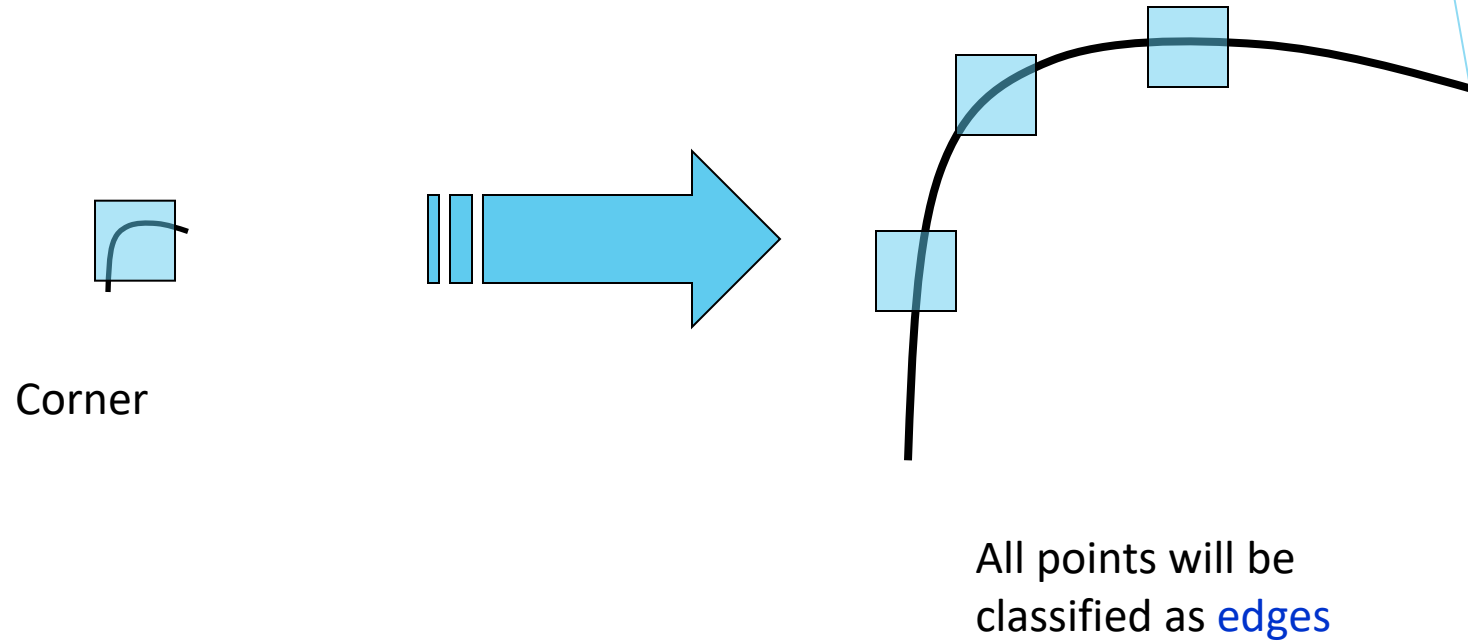
Image rotation



Ellipse rotates but its shape (i.e. eigenvalues) remains the same

Corner response R is invariant w.r.t. rotation and corner location is covariant

Scaling

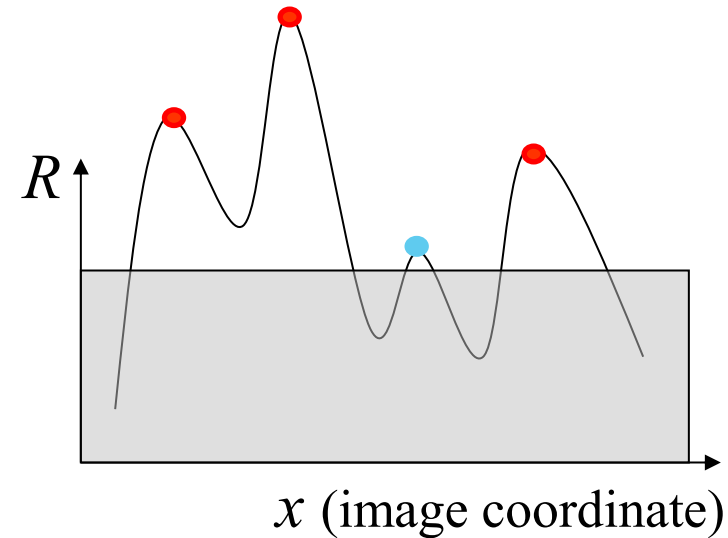
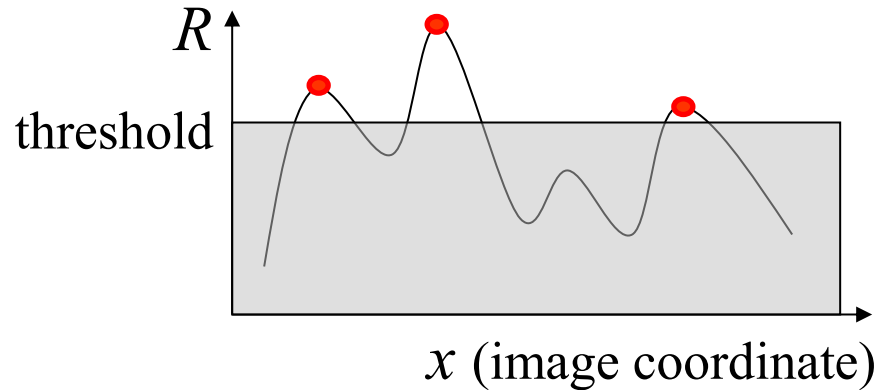


Not invariant to scaling

Affine intensity change

✓ Only derivatives are used $\Rightarrow$ invariance to intensity shift $I \rightarrow I + b$

✓ Intensity scale: $I \rightarrow a I$



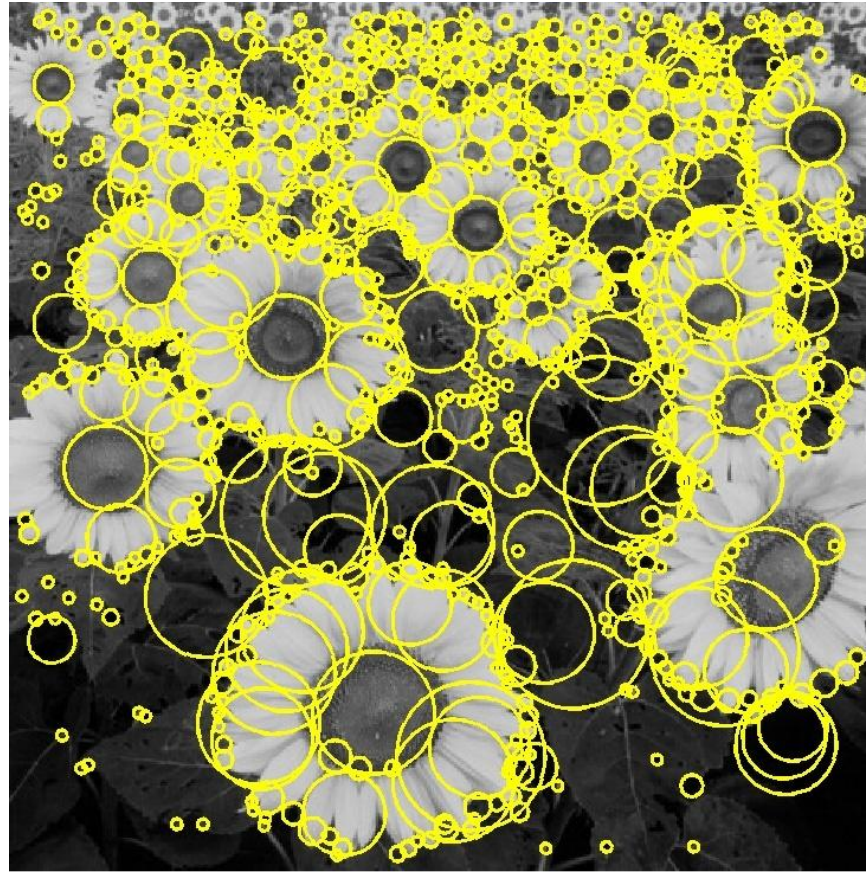
Partially invariant to affine intensity change

What about internal structure

- ▶ Edges & Corners convey boundary information
- ▶ What about interior texture of the object?

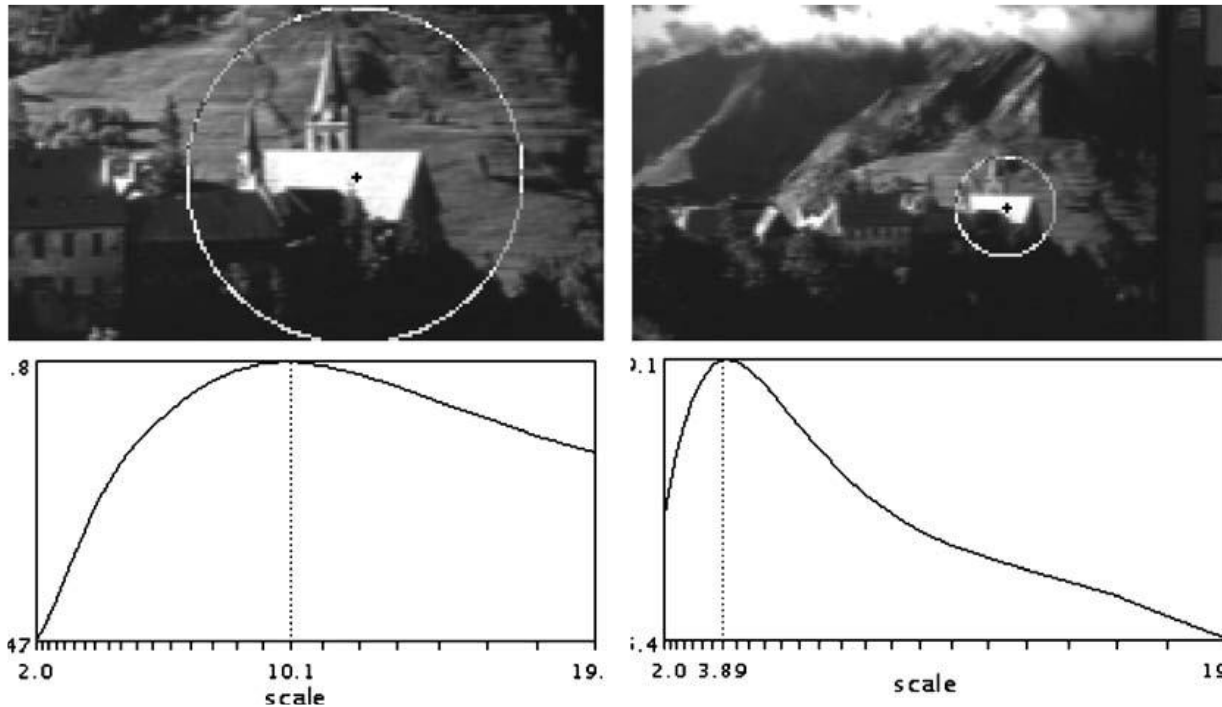


Blob detection with scale selection



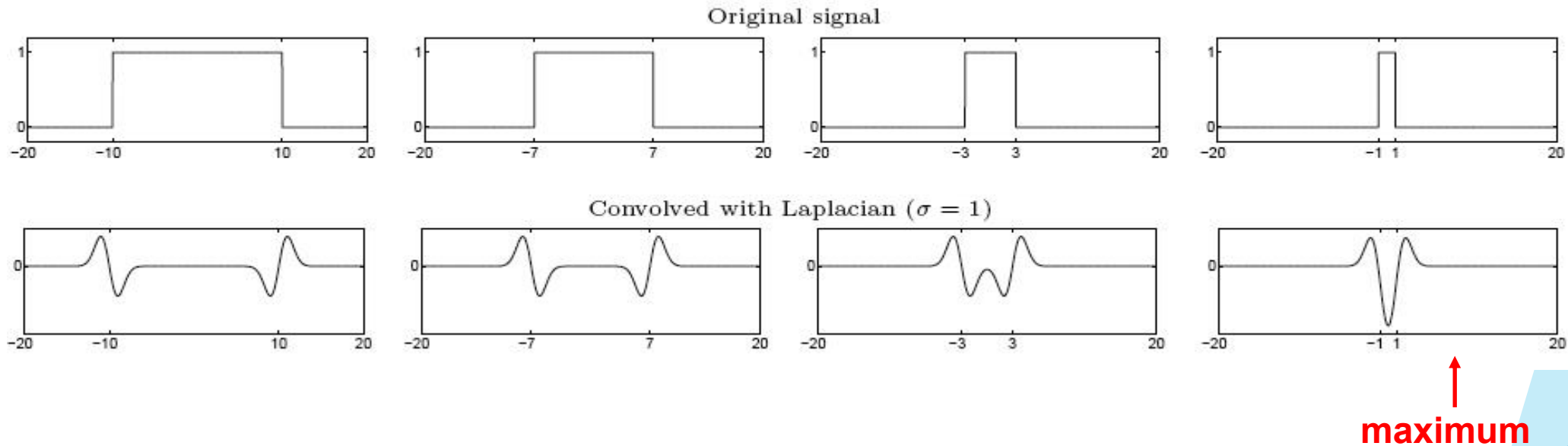
Achieving scale covariance

- Goal: independently detect corresponding regions in scaled versions of the same image
- Need *scale selection* mechanism for finding characteristic region size that is *covariant* with the image transformation



From edges to blobs

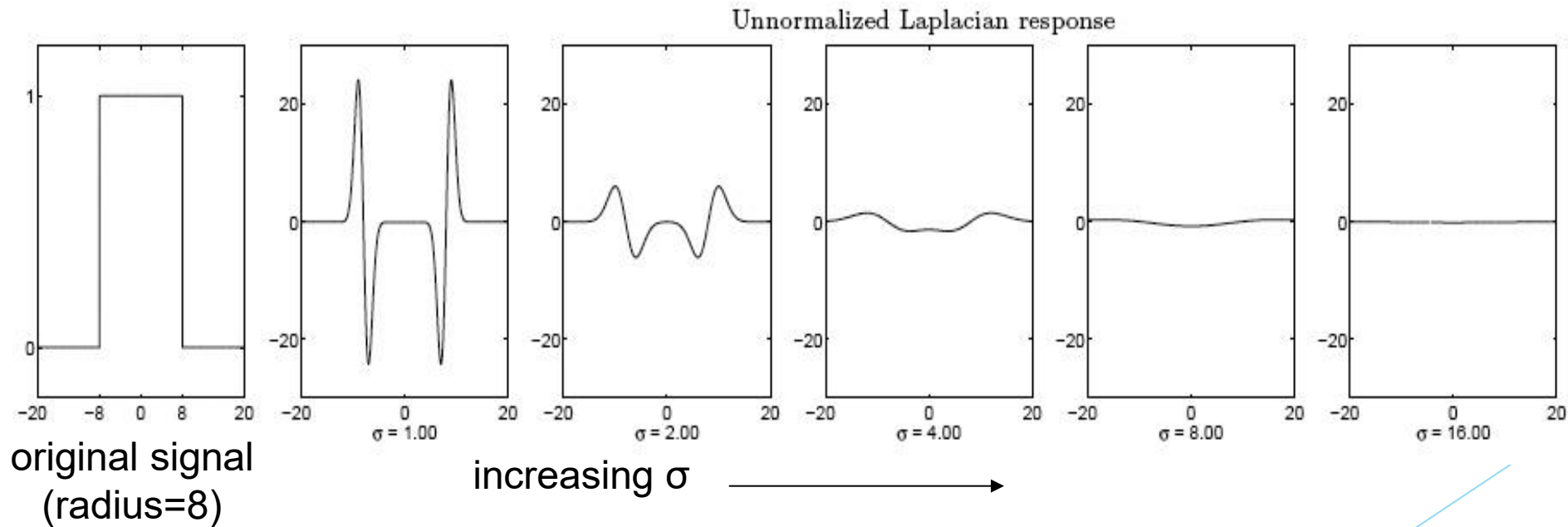
- Edge = ripple
- Blob = superposition of two ripples



Spatial selection: the magnitude of the Laplacian response will achieve a maximum at the center of the blob, provided the scale of the Laplacian is “matched” to the scale of the blob

Scale selection

- We want to find the characteristic scale of the blob by convolving it with Laplacians at several scales and looking for the maximum response
- However, Laplacian response decays as scale increases:



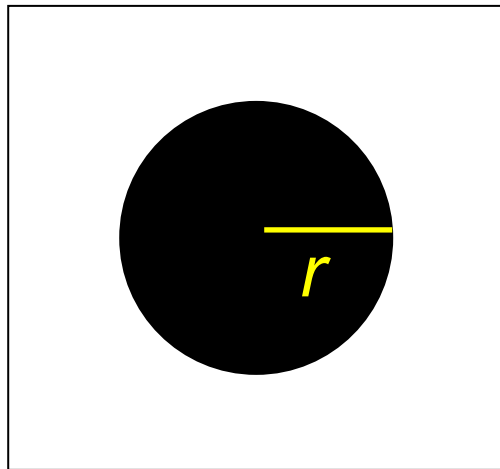
Scale normalization

- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2

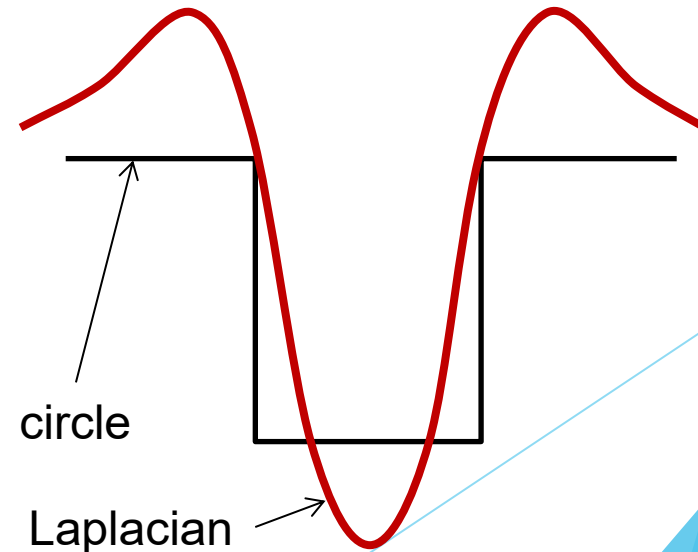
Scale selection

- To get maximum response, the zeros of the Laplacian have to be aligned with the circle
- Zeros of Laplacian is given by (up to scale):

- Therefore, the maximum response occurs at $\left(1 - \frac{x^2 + y^2}{2\sigma^2}\right) = 0$
 $\sigma = r / \sqrt{2}.$

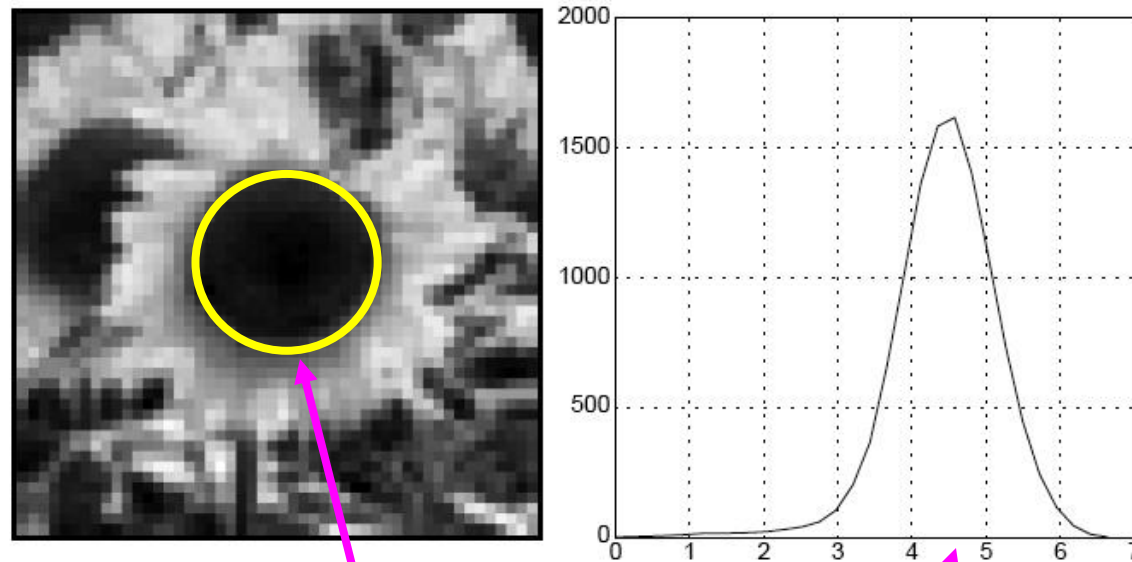


image



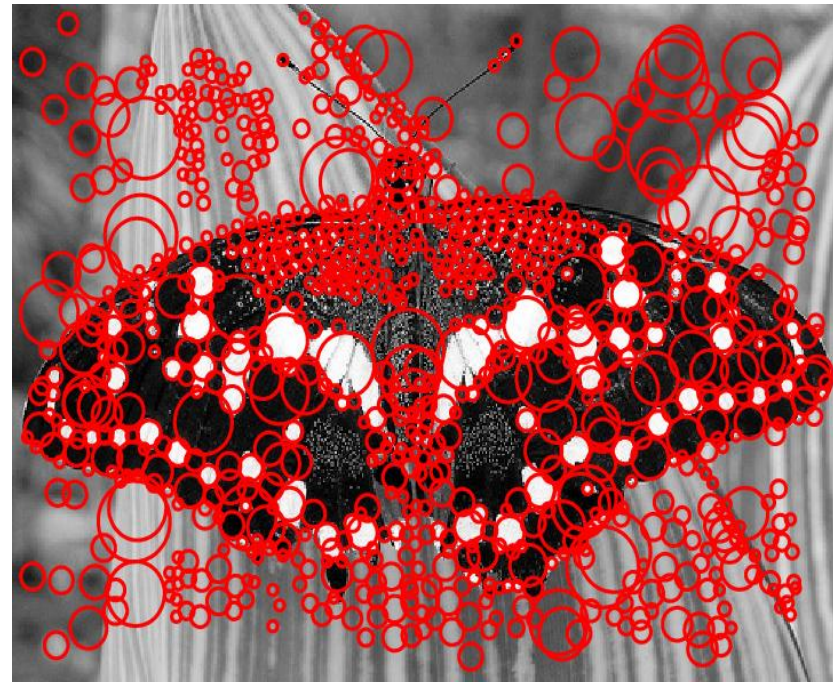
Characteristic scale

- We define the characteristic scale of a blob as the scale that produces peak of Laplacian response in the blob center



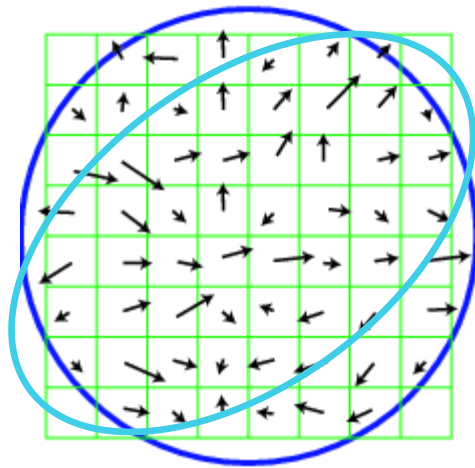
characteristic scale

Scale-space blob detector: Example



Affine adaptation

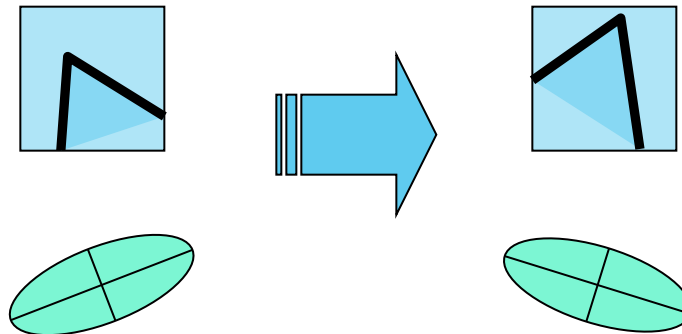
- Problem: the second moment “window” determined by weights $w(x,y)$ must match the characteristic shape of the region
- Solution: iterative approach
 - ▶ Use a circular window to compute second moment matrix
 - ▶ Perform affine adaptation to find an ellipse-shaped window
 - ▶ Recompute second moment matrix using new window and iterate



Descriptors Invariant to Rotation

- ▶ **Harris corner response measure:**
depends only on the eigenvalues of the matrix M

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$



Object matching



template

2D image

The Scale-Invariant Feature Transform (SIFT) algorithm is a widely used method in computer vision for detecting and describing local features in images. It was introduced by David Lowe.

The scale-invariant feature transform (SIFT)

- ▶ SIFT keypoints of objects are first extracted from a set of reference images and stored in a database. An object is recognized in a new image by individually comparing each feature from the new image to this database and finding candidate matching features based on Euclidean distance of their feature vectors.
- ▶ From the full set of matches, subsets of keypoints that agree on the object and its location, scale, and orientation in the new image are identified to filter out good matches. The determination of consistent clusters is performed rapidly by using an efficient hash table implementation of the generalized Hough transform. Each cluster of 3 or more features that agree on an object and its pose is then subject to further detailed model verification and subsequently outliers are discarded. Finally the probability that a particular set of features indicates the presence of an object is computed, given the accuracy of fit and number of probable false matches. Object matches that pass all these tests can be identified as correct with high confidence.

Scale Invariant Feature Transform

David Lowe IJCV 2004

Basic idea:

- Take 16x16 square window around detected feature
- Compute edge orientation (angle of the gradient - 90°) for each pixel
- Throw out weak edges (threshold gradient magnitude)
- Create histogram of surviving edge orientations

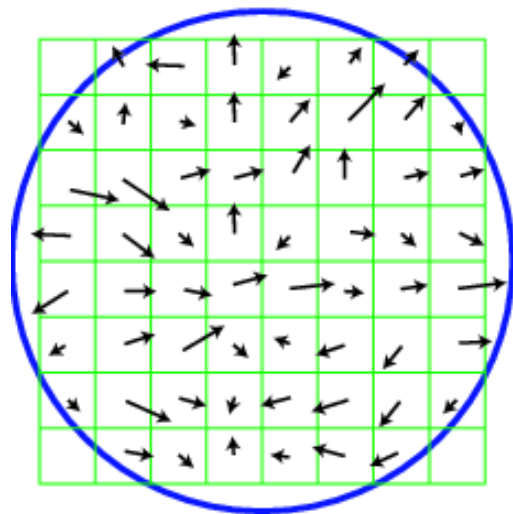
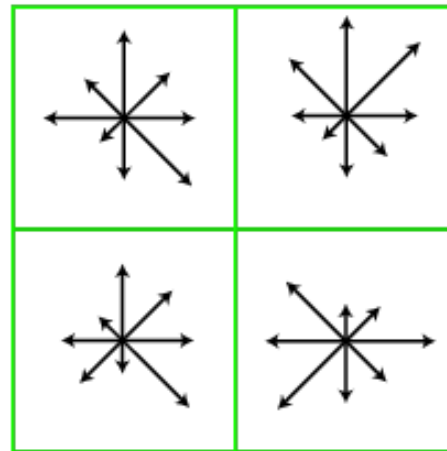
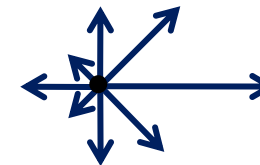
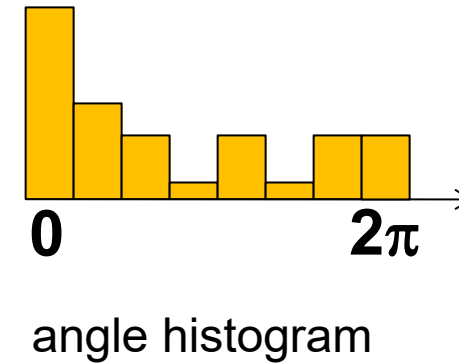


Image gradients



Keypoint descriptor

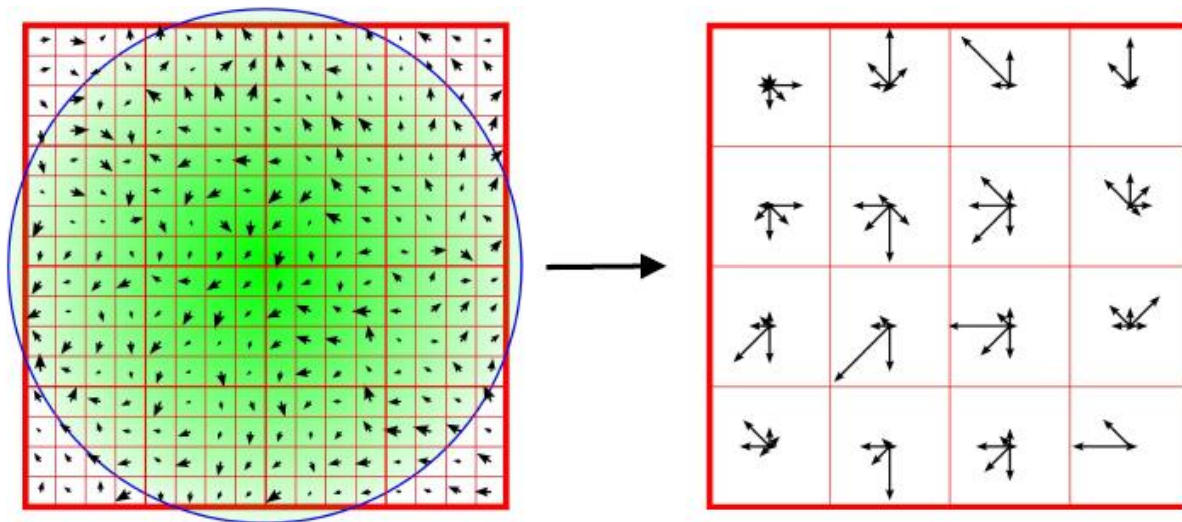


Adapted from slide by David Lowe

Former NYU faculty &
Prof. Ken Perlin's advisor

Orientation Histogram

- ▶ 4x4 spatial bins (16 bins total)
- ▶ Gaussian center-weighting
- ▶ 8-bin orientation histogram per bin
- ▶ $8 \times 16 = 128$ dimensions total
- ▶ Normalized to unit norm

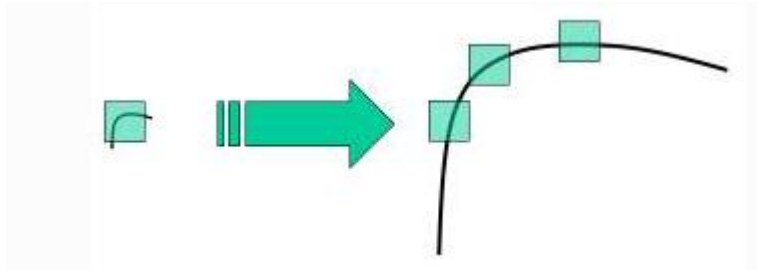


SIFT invariances

- ▶ Spatial binning gives tolerance to small shifts in location and scale
- ▶ Explicit orientation normalization
- ▶ Photometric normalization by making all vectors unit norm
- ▶ Orientation histogram gives robustness to small local deformations

SIFT Theory

A corner may not be a corner if the image is scaled. For example, check a simple image below. A corner in a small image within a small window is flat when it is zoomed in the same window.



So, in 2004, **D.Lowe**, University of British Columbia, came up with a new algorithm, Scale Invariant Feature Transform (SIFT) in his paper

SIFT Algorithm

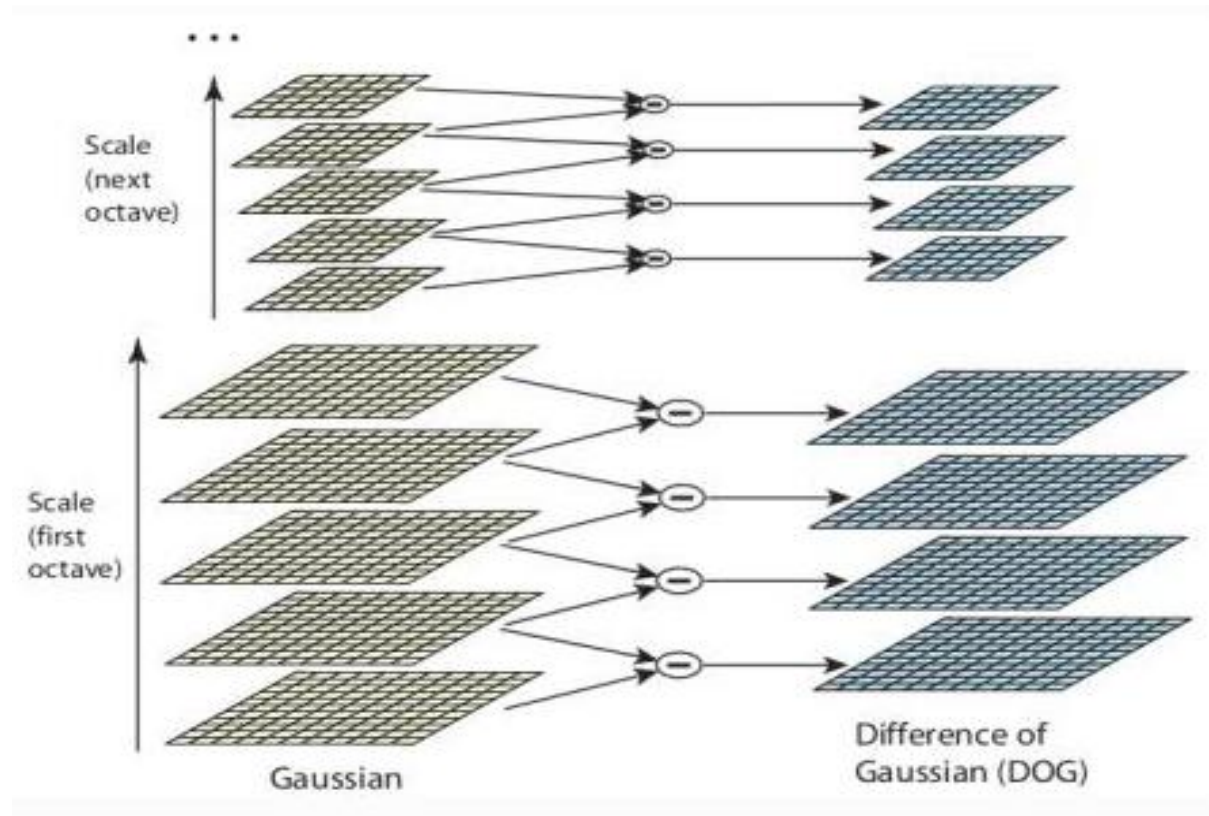
There are mainly 4 steps involved in SIFT algorithm.

1. Scale space Extrema Detection
2. Keypoint Localization
3. Orientation Assignment
4. Keypoint Descriptor

Keypoint Matching

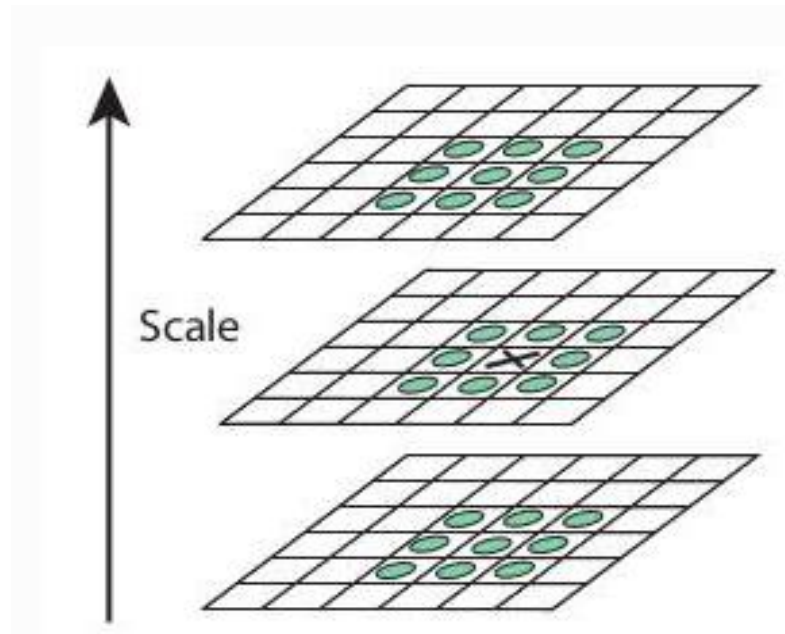
Scale Invariant Feature Transform

Scale space Extrema Detection



Scale Invariant Feature Transform

► Scale space Extrema Detection



In it, Laplacian of Gaussian is found for the image with various σ values. LoG acts as a blob detector which detects blobs in various sizes due to change in σ . In short, σ acts as a scaling parameter. For eg, in the above image, gaussian kernel with low σ gives high value for small corner while gaussian kernel with high σ fits well for larger corner. LOG is costly so SIFT uses DOG which is an approximation of LOG

Scale Invariant Feature Transform

2. Key point Localization

- ▶ Once potential keypoints locations are found, they have to be refined to get more accurate results. They used Taylor series expansion of scale space to get more accurate location of extrema, and if the intensity at this extrema is less than a threshold value (0.03 as per the paper), it is rejected. This threshold is called **contrastThreshold** in OpenCV

Scale Invariant Feature Transform

► Orientation Assignment

A neighbourhood is taken around the keypoint location depending on the scale, and the gradient magnitude and direction is calculated in that region. An orientation histogram with 36 bins covering 360 degrees is created. The highest peak in the histogram is taken and any peak above 80% of it is also considered to calculate the orientation. It creates keypoints with same location and scale, but different directions. It contribute to stability of matching

Scale Invariant Feature Transform

► Keypoint Descriptor

Now keypoint descriptor is created. A 16×16 neighbourhood around the keypoint is taken. It is divided into 16 sub-blocks of 4×4 size. For each sub-block, 8 bin orientation histogram is created. So a total of 128 bin values are available. It is represented as a vector to form keypoint descriptor. In addition to this, several measures are taken to achieve robustness against illumination changes, rotation etc.

Scale Invariant Feature Transform

► Keypoint Matching

Keypoints between two images are **matched by identifying their nearest neighbours**. But in some cases, the second closest-match may be very near to the first. It may happen due to noise or some other reasons. In that case, ratio of closest-distance to second-closest distance is taken. If it is greater than 0.8, they are rejected. It eliminates around 90% of false matches while discards only 5% correct matches, as per the paper

OpenCV Function

The SIFT constructors :

```
SIFT::SIFT(int nfeatures=0,  
           int nOctaveLayers=3,  
           double contrastThreshold=0.04,  
           double edgeThreshold=10,  
           double sigma=1.6)
```

OpenCV Function

Parameters:**nfeatures** - The number of best features to retain. The features are ranked by their scores (measured in SIFT algorithm as the local contrast)

nOctaveLayers - The number of layers in each octave. 3 is the value used in D. Lowe paper. The number of octaves is computed automatically from the image resolution.

contrastThreshold - The contrast threshold used to filter out weak features in semi-uniform (low-contrast) regions. The larger the threshold, the less features are produced by the detector.

edgeThreshold - The threshold used to filter out edge-like features. Note that the its meaning is different from the **contrastThreshold**, i.e. the larger the **edgeThreshold**, the less features are filtered out (more features are retained).

sigma - The sigma of the Gaussian applied to the input image at the octave #0. If your image is captured with a weak camera with soft lenses, you might want to reduce the number.

OpenCV Function

SIFT::operator()

Extract features and computes their descriptors using SIFT algorithm

```
SIFT::operator()(InputArray img,  
                 InputArray mask,  
                 vector<KeyPoint>& keypoints,  
                 OutputArray descriptors,  
                 bool useProvidedKeypoints=false)
```

OpenCV Function

Parameters:

img - Input 8-bit grayscale image

mask - Optional input mask that marks the regions where we should detect features.

keypoints - The input/output vector of keypoints

descriptors - The output matrix of descriptors. Pass `cv::noArray()` if you do not need them.

useProvidedKeypoints - Boolean flag. If it is true, the keypoint detector is not run. Instead, the provided vector of keypoints is used and the algorithm just computes their descriptors.

Practice code

```
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread('players.jpg', cv2.IMREAD_GRAYSCALE)

# Initialize the SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(image, None)

# Draw keypoints on the image
image_with_keypoints = cv2.drawKeypoints(
    image, keypoints, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Display the image
plt.imshow(image_with_keypoints, cmap='gray')
plt.axis('off')
plt.show()
```

Output



Input Image



Output Image

Input image



Fig. Input image

output

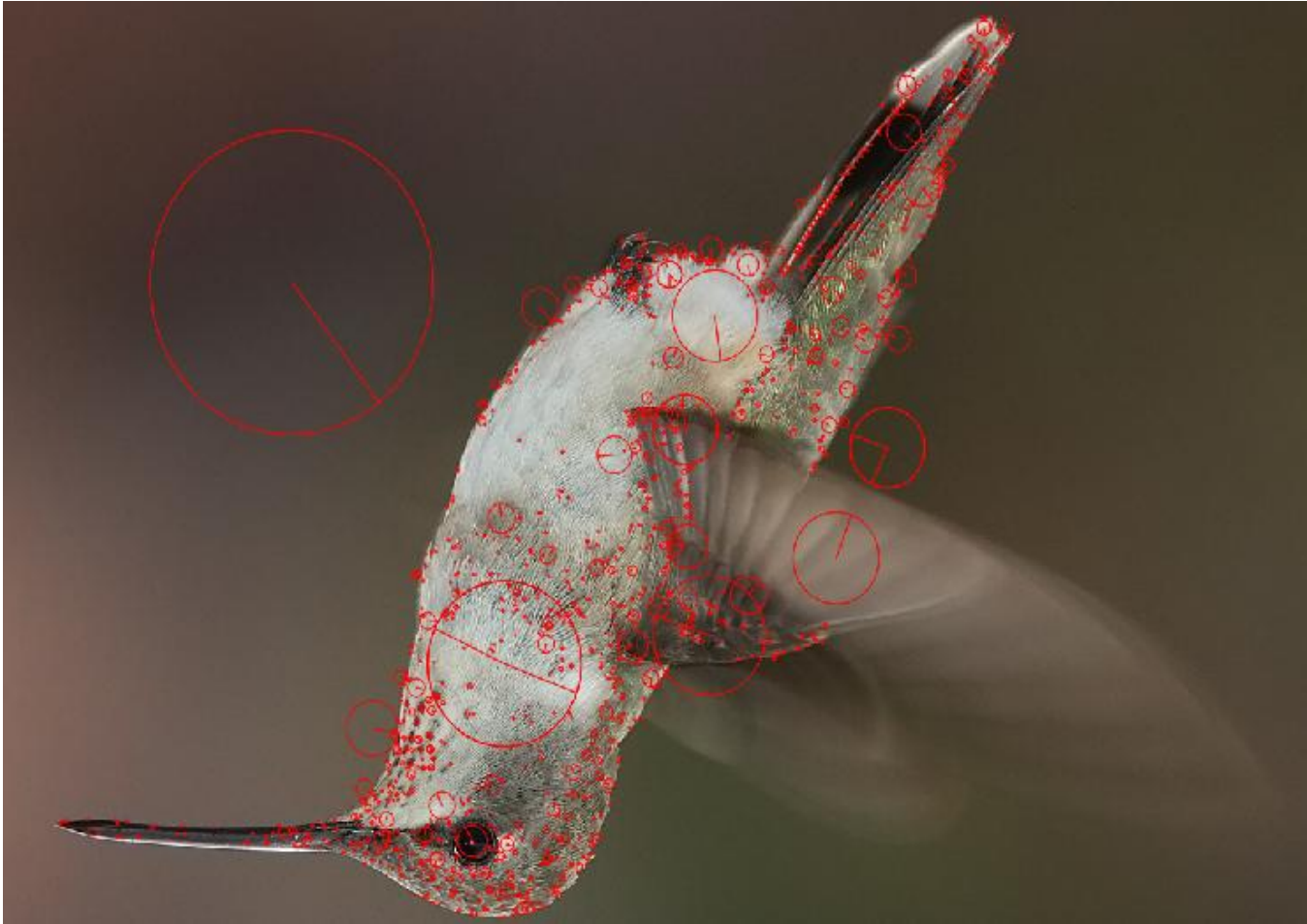


Fig. keypoint

Output

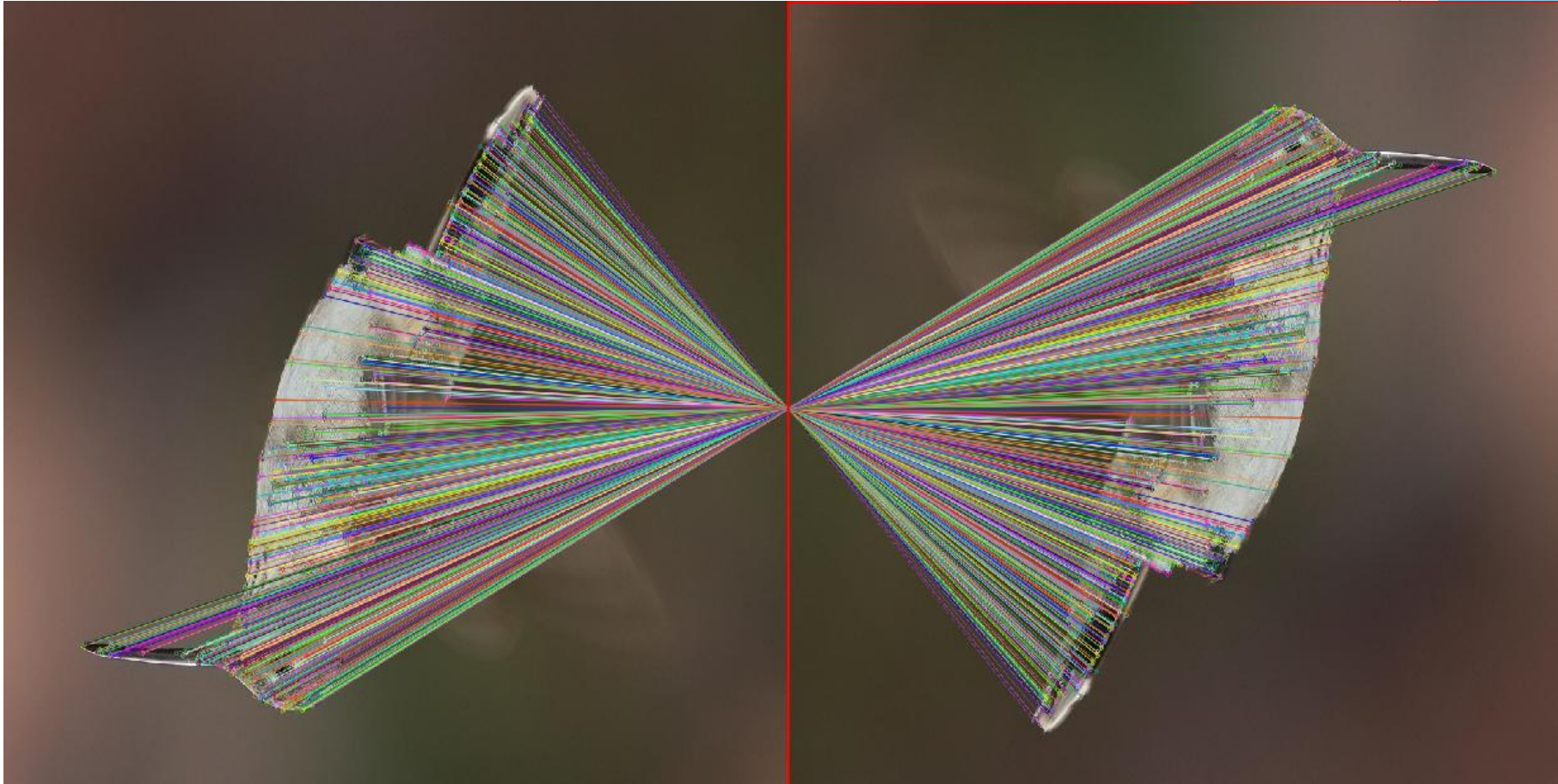


Figure: Keypoint matched

SIFT

```
import numpy as np
import cv2 as cv
img = cv.imread('home.jpg')
gray= cv.cvtColor(img,cv.COLOR_BGR2GRAY)
sift = cv.SIFT_create()
kp = sift.detect(gray,None)
img=cv.drawKeypoints(gray,kp,img)
cv.imwrite('sift_keypoints.jpg',img)
img=cv.drawKeypoints(gray,kp,img,flags=cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv.imwrite('sift_keypoints.jpg',img)
sift = cv.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
```

Output

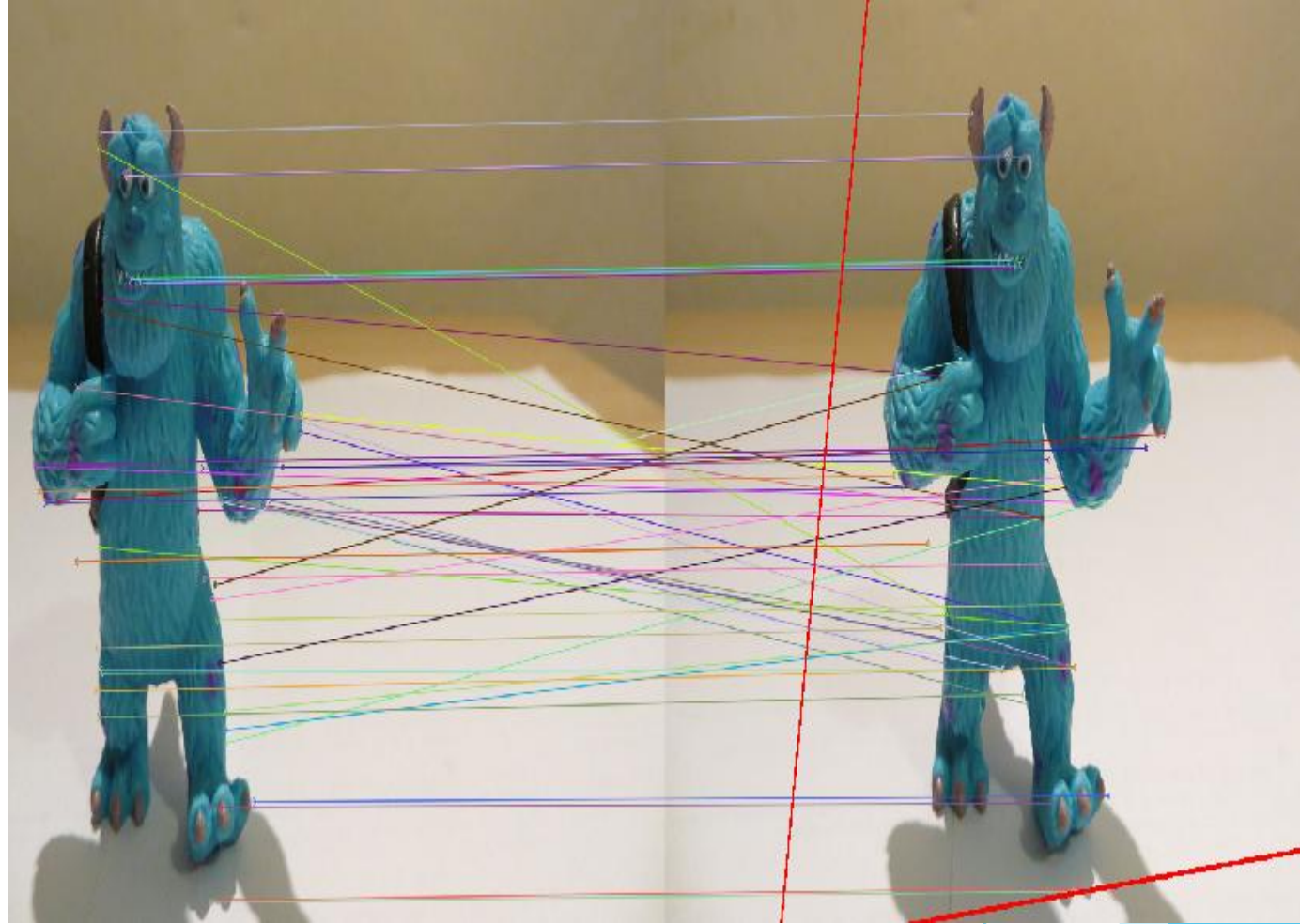


Fig. Key point Matching

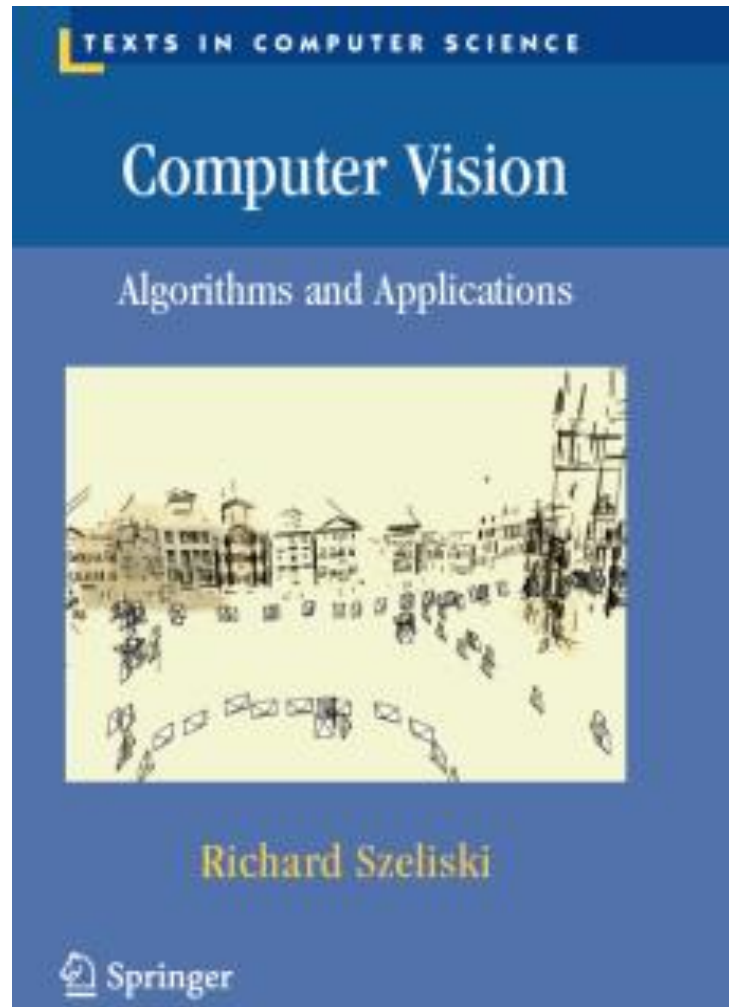
Exercise

Apply SIFT in video (.avi) file keypoint matching.

Reference

1. http://opencv-python-tutroals.readthedocs.org/en/latest/py_tutorials/py_feature2d/py_sift_intro/py_sift_intro.html
2. http://docs.opencv.org/modules/nonfree/doc/feature_detection.html
3. http://docs.opencv.org/doc/user_guide/ug_features2d.html
4. https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

Reference



Question



Thank you

